

Python Cheat Sheets (©mjye)

1 基础知识

1.1 数据类型

- `int` 783 0 -192 0b010 0o123 0xF3
- `float` 3.14 0.0 6.626e-34
- `str` 'one' "One\nTwo" 'X\tY\tZ'
- `bytes` b"toto\xfe\775"

1.2 容器类型

- `list` [1, 5, 9] ["x", 11, 8.9] ["one"] []
- `tuple` (1,5,9) 11,"y",7.4 ("one",) `tuple()`
- `dict` {"key": "value"} `dict(a=3,b=4,k=v) {}`
{1: "one", 3: "three", 2: "two", 3.14: "n"}
- `set` {"key1", "key2"} {1,9,3,0} `set()`

1.3 标识符

标识符用于变量、函数、模块、类等命名。按照如下格式定义：

- 必须以 [字母下划线开头]，跟着 [字母下划线数字]
- 不能使用关键字作为标识符
- 大小写敏感 (casesensitive)

☺ a _f_ x007 my_max BigOne_
☺ 666a a^b and for def

1.4 赋值

Python 的赋值过程是将变量 (标识符) 绑定值的过程。1. 计算右边的值，2. 然后左边的变量按顺序赋值。

- `y=2*sin(x)+1`
- `a=b=c=0`
- `a,b,c=1,2,3`
- `a,b=b,a`
- `a,*b=seq`
 `*a,b=seq`
- `a,*b=[1,2,3]`
 `→ a=1,b=[2,3]`

- `*a,b=[1,2,3]`
 `→ a=[1,2],b=3`
- `x+=2` `##=>x=x+2`
 `x-=3` `##=>x=x-3`
- `x=None`
- `del x`

1.5 输入输出

```
print(1,'2', a, b + c)
print(*contents, sep=" ", end="\n", file=sys.stdout)
s = input(" 输入说明:")
input 总是返回字符串，其他类型需要转换
```

1.6 类型转换 (type(expression))

- `int("12")` → 12
- `int("3f",16)` `##16 进制` → 63
- `round(3.1415,2)` `##2 位小数，默认 0 为整数` → 3.14
- `bool(x)`，当x为0、False、None、空序列²时为False，其他为True
- `chr(64)` → '@'，`ord('@')` → 64 (code ↔ char)
- `list("abcd")` → ['a', 'b', 'c', 'd']
- `"a b c".split()` → ['a', 'b', 'c']
- `','.join(['a', 'b', 'c'])` → "a,b,c"
- `"a,b,c".split(',')` → ['a', 'b', 'c']

1.7 索引 (strings/lists/tuples/bytes)

negative index	-5	-4	-3	-2	-1	
positive index	0	1	2	3	4	
1st=[10, 20, 30, 40, 50]						
negative slice	0	1	2	3	4	5
positive slice	-5	-4	-3	-2	-1	

索引切片: [start=1 : end=len(s) : step=1]
`lst[:-1]` → [10,20,30,40] `lst[::2]` → [10,30,50]
`lst[:]` → [10,20,30,40,50] `## 浅拷贝` `len(lst)` → 5

1.8 Boolean 逻辑

- 运算符: > < <= >= == !=
- `a and b`，如果`bool(a) == False`，返回 a；否则返回 b。
- `a or b`，如果`bool(a) == True`，返回 a；否则返回 b。³
- `not a`，返回`bool(a)`
- `0 < x > 3`等价于 `0 < x and x > 3`

1.9 条件与循环

```
if cond_1:
    statements
elif cond_2:
    ...
else:
    statements

while cond:
    statements

for var in sequence:
    statements
```

- 注意: if/else 只有第一个满足条件的语句块可以执行。
- `if bool(x)==True:↔if x:`
- `if bool(x)==False:↔if not x:`
- `a if cond else b` 类似于 C 的 `cond ? a : b`
例如: `a = 3 if 1 == 0 else 2` → `a = 2`
- `break`立刻退出，`continue`跳到下一个循环

2 容器的基本操作

- `len(c)` → 统计个数
- `min(c)`, `max(c)`, `sum(c)` → 注意: 对于字典和集合，这几个操作默认是针对 key 来操作
- `sorted(c)` → 排序
- `val in c` → 身份判断，是否在容器内 (相反的是`not in`)
- `all(c)` → 返回 True，如果 c 中所有的元素都是 True，否则返回 False
- `any(c)` → 返回 True，如果 c 至少有一个元素是 True，否则返回 False
- 对于顺序序列 (lists, tuples, strings, bytes...)，还有一下几个操作
 - `reversed(c)` → 逆序
 - `c.index(val)` → 索引位置
 - `c.count(val)` → 统计 val 在 c 中出现的次数
 - `c*n` → 容器元素复制 (浅拷贝)n 次
 - `c+c1` → 容器拼接 (类似 extend)

2.1 整数序列

- `range(/start/, end, /step/)`
start 默认为 0，end 不包含在序列中，step 可以是正负数，默认为 1
- `range(4)` → 0 1 2 3 `range(2,10,3)` → 2 5 8
- `range(3,8)` → 3 4 5 6 7 `range(8,2,-3)` → 8 5

¹key 必须要 hashable; 例如: 列表不能作为 key..

²{ }; { }; tuple(); set(); [].

³and 和 or 都会逻辑短路.

2.2 字符串操作

- `s.startswith(prefix)` 字符串开头子串
- `s.endswith(suffix)` 字符串结尾子串
- `s.is...()` eg: `s.isalpha()`, `s.isspace()`, `s.isdigit()`
- `s.upper()`, `s.lower()`, `s.title()`, `s.swapcase()`
- `s.split(/sep/)`, `s.join(seq)`
- 字符串格式化
`<str> = f'{{<el_1>}}, {{<el_2>}}'`
`<str> = '{{}}, {{}}'.format(<el_1>, <el_2>)`

2.3 Lists

- `lst.append(val)` 在末尾添加元素 `val`
- `lst.extend(seq)` 在末尾添加 `seq` 中的元素
- `lst.insert(idx, val)` 在位置 `idx` 插入 `val`
- `lst.remove(val)` 删除第一个 `val`
- `lst.pop(/idx/)` 删除并返回 `idx` 的元素 (默认 `last`)
- `lst.sort()` `lst.reverse()` 排序/逆序 (in place)
- 列表推导式

```
>>> [i for i in range(10)]
Out: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [i for i in range(10) if i%2== 0]
Out: [0, 2, 4, 6, 8]
>>> [(i,j) for i in range(1,3) for j in range(4,6)]
Out: [(1, 4), (1, 5), (2, 4), (2, 5)]
>>> [i*2 for i in [j for j in range(4)]]
Out: [0, 2, 4, 6]
```

2.4 Dictionaries

- `d[key]=value` `d[key] → value`
- `d.clear()` `del d[key]`
- `d.update(d2) → update(已存在)/add (不存在)`
- `d.keys()` `d.values()` `d.items()`
- `d.pop(key) → value` `d.popitem() → (key,value)` `d.get(key) → value`
- *# Creates a dict from coll. of key-value pairs.*

```
>>> d = dict(a=1,b=2,c=3)
Out: {'a': 1, 'b': 2, 'c': 3}
>>> d = dict({"a":1,"b":2,"c":3})
Out: {'a': 1, 'b': 2, 'c': 3}
>>> d = dict(zip(['a','b','c'], [1,2,3]))
Out: {'a': 1, 'b': 2, 'c': 3}
# <dict> = dict.fromkeys(keys [, value])
d = dict.fromkeys(['a','b','c'])
Out: {'a': None, 'b': None, 'c': None}
```
- 排序

```
>>> d = {"ye":10, "meng":20, "jie":3}
# 按照 key 排序
>>> sorted(d, key=lambda item:item[0])
Out: [('jie', 3), ('meng', 20), ('ye', 10)]
# 按照 value 排序
>>> sorted(d, key=lambda item:item[1])
Out: [('jie', 3), ('ye', 10), ('meng', 20)]
```

2.5 Sets

- `s|t` 并集 `s&t` 交集 `s-t` 差集
- `<` `<=` `>` `>=` 包含关系判断

```
>>> a = {1,2,3}
>>> b = {2,3,4}
>>> a | b
Out: {1,2,3, 4}
>>> a & b
Out: {2,3}
>>> a - b
Out: {1}
>>> b - a
Out: {4}
>>> a ^ b
Out: {1,4}
```

3 函数

```
def fun(x, y, z):
    """ 文档说明 """
    # statements blocks

    return res # 如果没有返回, 则 return None
```

复杂函数声明

```
def fun(x,y,z,*args, a = 1, b = 2, **kwargs)
*args 返回元组, **kwargs 返回字典
```

3.1 常用函数

```
from functools import reduce
#<iter> = enumerate(seq, start=0)
#[(0,'a'), (1,'b')]
>>> list(enumerate(['a','b']))
#map -> [f(ai) for ai in a]
# (1, 2, ..., 10)
<iter> = map(lambda x: x + 1, range(10))

# filter -> [ai, if f(ai) ]
```

```
# (6, 7, 8, 9)
>>> <iter> = filter(lambda x: x > 5, range(10))

#reduce -> f(f(f(a0,a1),a2),a3).....
# 45
>>> <obj> = reduce(lambda out, x: out+x, range(10))
```

4 文件读写

```
with open("file.txt", mode="wr", encoding="utf8") as f:
    #processing line by line
    for line in f.readlines():
        #processing a character in each line
        for c in line:
            #do something
```

- `f.read(/n/)` 如果 `n` 不指定, 读到文件结尾
- `f.readlines(/n/)` 返回每一行的列表, 如果 `n` 不指定, 返回所有行
- `f.readline()` 返回一行
- `f.write("something")`
- `f.writelines(list of lines)`
- `f.close()` `f.seek(postion)` `f.tell()` *##-> postion*

5 面向对象

```
class Name>:
    #constructor overloading
    def __init__(self, a=None, b=None):
        self.a = a #public
        self._b = b #private
    def __repr__(self):
        class_name = self.__class__.__name__
        return f'{{class_name}}({{self.a!r}})'
    def __str__(self):
        return str(self.a)

# 实例方法 实例参数 self
def instance_method(self):
    return self.a

# 类方法 类参数 cls
@classmethod
def get_class_name(cls):
    return cls.__name__

# 静态方法 无实例参数 self 和类参数 cls
@staticmethod
```

```

def static_method():
    #do something else

# 继承
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, score):
        super().__init__(name, age)
        self.score = score

# 多继承
class A: pass
class B: pass
class C(A, B): pass

# 属性 getter setter
@property
def b(self):
    return self._b

@a.setter
def b(self, value):
    self._b = value

# Duck Types
def __eq__(self, other):
    if isinstance(other, type(self)):
        return self.a == other.a
    return NotImplemented

def __lt__(self, other):
    if isinstance(other, type(self)):
        return self.a < other.a
    return NotImplemented

# 迭代器
def __next__(self):
    self.i += 1
    return self.i
def __iter__(self):
    return self
def __contains__(self, el):
    return el in self.a
def __len__(self):
    return len(self.a)

```

```

def __getitem__(self, i):
    return self.a[i]
#hashable
def __hash__(self):
    return hash(self.a)

```

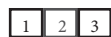
- repr() 的返回值应该是明确的, str() 的返回值应该是可读的。
- 如果只定义了 repr(), 那么它也将用于 str()。
- 如果 eq() 方法没有被覆盖, 它返回'id(self) == id(other)', 这和'self is other' 是一样的。

	Iterable	Collection	Sequence	abc.Seq
iter()	REQ	REQ	Yes	Yes
contains()	Yes	Yes	Yes	Yes
len()		REQ	REQ	REQ
getitem()			REQ	REQ
reversed()			Yes	Yes
index()				Yes
count()				Yes

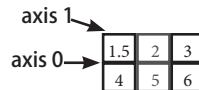
6 Numpy

```
import numpy as np
```

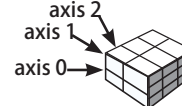
1D array



2D array



3D array



6.1 创建 Arrays

```

>>> a = np.array([1,2,3])
>>> b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
>>> c = np.array([(1.5,2,3), (4,5,6)],
                 [(3,2,1), (4,5,6)], dtype = float)
#0 数组
>>> np.zeros((3,4))
#1 数组
>>> np.ones((2,3,4), dtype=np.int16)
# 整数数组
>>> d = np.arange(10,25,5)
# 浮点数
>>> np.linspace(0,2,9)
# 常数数组
>>> e = np.full((2,2),7)
# 单位矩阵 2x2
>>> f = np.eye(2)
# 随机
>>> np.random.random((2,2))

```

```

# 空数组
>>> np.empty((3,2))

```

6.2 操作

数组操作是 broadcast

- 数组属性 a.shape len(a) a.ndim a.size a.dtype a.dtype.name a.astype(int)
- 数据类型 np.int64 np.float32 np.complex np.bool np.object
- 基本操作 a+b a-b a/b a*b np.multiply(a,b) np.exp(b)
- 统计 a.sum() a.min() a.max(axis=1) a.cumsum(axis=0) a.mean() a.median() a.corrcoef() np.std(b)
- 比较 a == b, a > 2 ...
- 复制 np.copy(a) a.copy() a.view()
- 排序 a.sort() a.sort(axis=0)

转置

```

>>> i = np.transpose(b)
>>> i.T

```

改变数组维度

```

>>> b.ravel() # 降为 1 维, 返回 view
>>> b.flatten() # 降为 1 维, 返回 copy
>>> g.reshape(3,-2) # 重排, 行优先
# 添加/删除元素

```

```

>>> h.resize((2,6))
>>> np.append(h,g)
>>> np.insert(a, 1, 5)
>>> np.delete(a,[1])

```

数组拼接

```

# 第一个参数元组 (a,d), 第二参数维度
>>> np.concatenate((a,d),axis=0)
# (row-wise)

```

```
>>> np.vstack((a,b))
```

```
>>> np.r_[e,f]
```

#(column-wise)

```
>>> np.hstack((e,f))
```

```
>>> np.column_stack((a,d))
```

```
>>> np.c_[a,d]
```

分割数组

```
>>> np.hsplit(a,3)#col 方向
```

```
>>> np.vsplit(c,2)#row 分割
```

7 Matplotlib

```

from matplotlib import pyplot as plt
import numpy as np
#step1 准备数据
#linspace(start, end, sampling)

```

```
# 均匀分布的 100 个数 endpoint=False
x = np.linspace(-np.pi, np.pi, 100)
y = np.sin(x)
# 设置图形对象: 窗口
plt.figure('FIG', # 图形对象名称 窗口左上角显示
figsize = (8, 6), # 窗口大小, 英寸
dpi = 120, # 分辨率
facecolor = 'lightgray', # 背景色
)
# 绘制 等价于 plt.plot(x,y, 'b--*')
plt.plot(x, y,
linewidth=3, # 线条粗细
color='blue', # 线条颜色
linestyle='--', # 线条风格
marker="*", # 点样式
label=r'$\sin(x)$'
)
# 坐标轴标签
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("f = sin(x)", fontsize=14)

# 设置坐标轴边界 xlim ylim
plt.xlim(x.min() * 1.1, x.max() * 1.1)
plt.ylim(y.min() * 1.1, y.max() * 1.1)

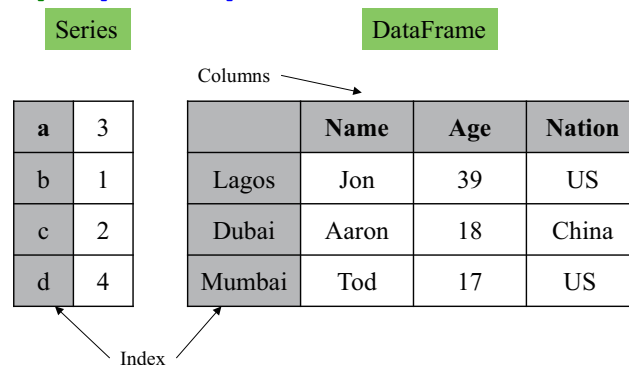
# 显示坐标刻度
plt.xticks([-np.pi, -np.pi/2, 0,
np.pi/2, np.pi*3/4, np.pi],
[r'$-\pi$', r'$-\frac{\pi}{2}$', r'$0$',
r'$\frac{\pi}{2}$', r'$\frac{3\pi}{4}$', r'$\pi$'])
plt.yticks([-1, -0.5, 0.5, 1])

# 获取当前坐标轴
ax = plt.gca()
# 设置零点在坐标轴的位置, set_position(tuple)
ax.spines['left'].set_position(('data',0))
ax.spines['bottom'].set_position(('data',0))
# 隐藏 top 和 right 边框
ax.spines['right'].set_visible(False)
ax.spines['top'].set_visible(False)
# 网格 grid 参数
plt.grid(color = 'y', linestyle=':', linewidth=1)
# 设置图例 legend
# loc 顺序: “先上中下, 再左中右” 默认上左
# shadow default: False
plt.legend(loc='upper left', shadow=False, fontsize=12)
# 保存图片
# 一定要在 plt.show() 之前保存
```

```
plt.savefig("fig.png")
# 显示图片
plt.show()
```

8 Pandas

```
import pandas as pd
```



- Series 带有索引的一维数组
- DataFrame 有索引的二维数组，每一列都是 Series

```
>>> s=pd.Series([3,1,2,4], index=['a','b','c','d'])
>>> data={'Name': ['Jon', 'Aaron', 'Tod'],
          'Age': ['39', '28', '17'],
          'Nation': ['US', 'China', 'US']}
>>> df=pd.DataFrame(data,
                    index=['Lagos', 'Dubai', 'Mumbai'])
```

```
>>> s['a']
Out: 3
>>> df[1:]
Out:
      Name  Age Nation
Dubai  Aaron   28  China
Mumbai  Tod   17   US
```

8.1 基本信息

- df.shape (rows, columns)
- df.index 行索引
- df.columns 列索引
- df.info() DataFrame 的基本信息
- df.count() non-NAN 数目

8.2 统计

- df.sum()

- df.cumsum()
- df.min() / df.max() 最大/小值
- df.idxmin() / df.idxmax() 最大/小值索引
- df.describe() 统计摘要
- df.mean() / df.median() 均值/中值

8.3 操作

```
# 通过 position
#df.iloc([row], [column])
>>> df.iloc([0], [0]) # df.iat([0], [0])
Out: "Jon"
# 通过 label
>>> df.loc([0], ["Name"]) # df.at([0], ["Name"])
Out: "Jon"
# 获取一行
>>> df["Name"]
# 获取一行
>>> df.loc['Lagos']
# 获取多行
>>> df["Name", "Age"]
# 获取多行
>>> my_df.loc[['Lagos', 'Mumbai']]
# 添加新列
>>> df['Year']=pd.Series(['2016', '2017', '2018'],
                        ['Lagos', 'Dubai', 'Mumbai'])
# 删除列
>>> df.drop('Year', axis=1)
# 删除行
>>> df.drop('Dubai', axis=0)
# 筛选
>>> df[df["Age"] > 30]
# 分组
>>> df.groupby("Nation")
# 画图
>>> df.plot(kind='bar', y='Age')
```

8.4 读写

```
>>> pd.read_csv("file.csv", header=None, nrow=10)
>>> pd.to_csv("outfile.csv")
# 单个 excel
>>> pd.read_excel("file.xlsx")
>>> pd.to_excel("outfile.xlsx", sheet_name="Sheet1")

# 多个 excel
>>> xlsx = pd.ExcelFile("file.xlsx")
>>> df = pd.read_excel(xlsx, "Sheet1")
```